

4

Unit - III

ESTIMATION FOR SOFTWARE PROJECTS

Syllabus

Estimation for Software Projects : The Project Planning Process, Defining Software Scope and Checking Feasibility, Resources management, Reusable Software Resources, Environmental Resources, Software Project Estimation, Decomposition Techniques, Software Sizing, Problem-Based Estimation, LOC-Based Estimation, FP-Based Estimation, Object Point (OP) -based estimation, Process-Based Estimation, Process-Based Estimation, Estimation with Use Cases, Use-Case-Based Estimation, Reconciling Estimates, Empirical Estimation Models, The Structure of Estimation Models, The COCOMO II Mode, Preparing Requirement Traceability Matrix.

4.1 The Project Planning Process

- The project planning is an important activity performed by the project managers in order to estimate the following entities for their optimum use :
 - The resources required for the project under development.
 - The cost of the project and
 - The schedule for in-time completion of the project.
- All these estimates are made in the stipulated amount of time in the starting of the project itself. Later on these estimates can be updated time to time. In order to achieve the objectives of project planning, following activities are carried out :
 - Defining software scope
 - Estimating resources.

4.2 Defining Software Scope and Checking Feasibility

University Question

Q. What is the need for defining a software scope ?

SPPU - Dec. 18, 4 Marks

- A software project manager is normally confused with a condition in the beginning of a software engineering project i.e. quantitative estimates and an organized plan. They are required and necessary but proper information is not available.
- A detailed analysis of software requirements may provide all the necessary information for estimates, but analysis often takes too much time to complete, even weeks or months. Requirements may be changing regularly as the project proceeds.

- Therefore, the developer and the manager should examine the product and the problem. It is intended to solve at the beginning of the project itself. At a minimum, the scope of the product must be established and bounded.
- The first software project management endeavor is the finding out of software scope. Scope can be defined by answering the following simple questions :
 - **Context** : How the software system can be developed to fit into a large system or some business context and what are the constraints that should be considered as a result of the context considered ?
 - **Information objectives** : What data objects are produced as output through the software developed from customer's point of view ? And what data objects will be used for the input ?
 - **Function and performance** : What are the functions that the software should perform in order to translate input into output ? Is there any special performance characteristic that is supposed to be addressed ?
- There should be a clear and comprehensive software project scope defined and that should be understandable at all the levels (i.e. the management and technical levels).
- Generally the software scope describes the following :
 - Performance
 - Function
 - The data and control used to define the constraints
 - Reliability and
 - Interfaces.
- Functions are described in the statement of scope are assessed and evaluated to provide details in estimation. The cost and schedule are two important parameters and they are functionality oriented.

4.2.1 Obtaining Information Necessary for Scope

- In start of a development process, it is always good and necessary to define the scope of the project. The frequent communication between the customer and developer builds the foundation for defining scope properly. The frequent meetings between the customer and developer are very important to bridge the gap in all sorts of communication.
- Following are the set of questions that focuses on the goals and objectives :
 - Know the person behind the request for this work.
 - See who will use the solution?
 - What will be the commercial benefit of a solution?
 - Is there any other resource of solution?
- These questions clear the understanding of problems and customers can raise their doubts.

4.2.2 Feasibility

- After defining and identifying the scope of the project, the developer should look at the scope and should estimate the feasibility whether the project is feasible or not.
- Otherwise, after spending lot of time working on the project, it is found that the scope is not feasible to develop. Hence it is always necessary to check the feasibility in the starting of the development process itself to avoid the losses.

4.3 Estimating Resources : Resource Management

University Question

Q. What are the categories of software Engineering resources ?

SPPU - Dec. 18, 4 Marks

- The second planning task is estimation of the resources required to accomplish the software development effort.
- The Fig. 4.3.1 depicts the three major categories of software engineering resources : people, reusable software components, and the development environment.
- Each resource is specified with four characteristics :
 - Description of the resource;
 - A statement of availability;
 - Time when the resource will be required;
 - Duration of time that resource will be applied.

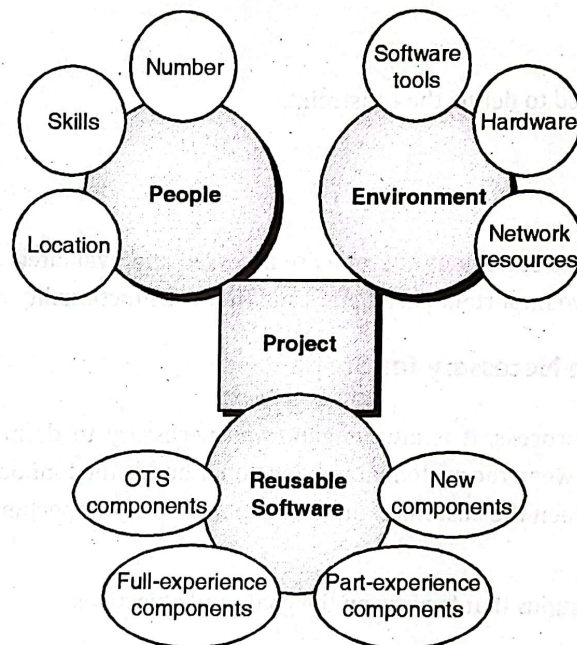


Fig. 4.3.1 : Project Resources

4.3.1 Reusable Software Resources

- Reusability is an important software building block that must be catalogued for easy reference, standardized for easy application, and validated for easy integration. Four software resources have been suggested as categories that should be considered as planning proceeds :
 - **Off-the-shelf components** : The reusability is an important feature and thus the existing software can be purchased from a third party to use it in the current project. Usually the COTS i.e. "commercial off the shelf" are purchased. They can be completely validated after use in the current project.
 - **Full-experience components** : The existing components had undergone through all phases of software development life cycle i.e. specifications, designs, code, and testing. Thus these full-experienced components can be used in the current project.



- **Partial-experience components** : Some components can be used directly, but they need some modification before they can be used. They are referred as partial-experience components. They had also gone through all phases of software development life cycle i.e. specifications, designs, code, and testing. Still they require modification.
- **New components** : In addition to all these components, some new components may also be built as per the demands of the current project.
- Thus the evaluation of all these alternatives is conducted.

4.3.2 Environmental Resources

- The Software Engineering Environment (SEE) supports the software project. The SEE normally supports all the engineering activities of the development process.
- Similarly the hardware also provides a platform that supports all the software tools needed to complete an application by using some good engineering practices.
- While designing and developing a computer based system, the development team uses all the hardware elements designed by other development team. All these resources are referred as environmental resources.

4.4 Software Project Estimation

University Question

Q. What is need of project estimation?

SPPU - May 18, May 19, Dec. 19, 6 Marks

- Software cost and effort estimation will never be an exact science. Too many variables are there like human, technical, environmental, political that can affect the ultimate cost of software and effort applied to develop it.
- However, software project estimation can be transformed from a black art to a series of systematic steps that provide estimates with acceptable risk.
- To achieve reliable cost and effort estimates, a number of options arise :
 - Delay estimation until late in the project (obviously, we can achieve 100% accurate estimates after the project is complete!).
 - Base estimates on similar projects that have already been completed.
 - Use relatively simple decomposition techniques to generate project cost and effort estimates.
 - Use one or more empirical models for software cost and effort estimation.

4.5 Decomposition Techniques

University Question

Q. What is project decomposition? What are the work tasks for communication, process using process decomposition?

SPPU - May 18, 6 Marks

- Software project estimation is a form of problem solving, and in most cases, the problem to be solved (i.e., developing a cost and effort estimate for a software project) is too complex to be considered in one piece.



- For this reason, we decompose the problem, recharacterizing it as a set of smaller (and hopefully, more manageable) problems.
- We use following two types of Decomposition Techniques :
 - Problem decomposition and
 - Process decomposition

4.5.1 Problem Decomposition

- Problem decomposition is an activity that is present during software requirements analysis. The problem decomposition, sometimes called as **partitioning** or **problem elaboration**.
- During defining the scope of software, the problem is not fully decomposed. The decomposition activity is applied in following two important areas :
 - The functionality that is expected to be delivered and
 - The process that helps to deliver it
- Generally all the human beings prefer a divide-and-conquer scheme when they face any complex problem. It can be stated in this way that a complex problem is divided into smaller sub problems that can be managed very easily.
- This technique is often applied in the beginning of the project planning.
- During the evolution of the statement of scope, the first level of partitioning occurs naturally. For example, the project team members learn that the marketing team has communicate with potential customers and found that the following functions should be the part of automatic copy editing required :
 - Spell checking
 - Sentence grammar checking
 - Reference checking for large documents
 - Section and chapter reference validation for large documents.
- All these features represent a sub-function that is supposed to be implemented in software. Each feature can be further refined if the decomposition will make planning easier.

4.5.2 Process Decomposition

- An outline for project planning is ascertained by the process framework. It is adapted by allocating a task set that is appropriate to the project. The framework activities that characterize the software process are applicable to all software projects. The problem is to select the process model that is appropriate for the software to be engineered by a project team.
- The project manager should decide that which of the process model is most appropriate for :
 - The customers who request the product and the people who will do the work.
 - The characteristics of the software product.
 - The project environment in which the software team works.
- When a process model is selected, the team then defines a preliminary project plan based on the set of process framework activities. After establishing the preliminary plan, the process decomposition can be started. A complete plan must be created, that reflects the work tasks required to populate the framework activities.



- It is always recommended that a software team should be adaptable in selecting the software process model that is most appropriate for the project and all the software engineering tasks can be implemented once the process model is chosen.
- Usually small projects can be completed using the linear sequential approach.
- If time constraints are imposed very strictly then the problem might be heavily modularized into several small models and generally RAD model would be the correct choice.
- If the deadline is to be followed very strictly then it is fact that full functionality may not be delivered in the first version and in this scenario, an incremental approach would be most suitable approach.
- If projects encounter with following characteristics, then some other process models might be selected :
 - Uncertain requirements
 - Breakthrough technology
 - Difficult customers
 - Significant reuse potential.
- Once the process model has been finalized, the process framework is adapted to it. In all the cases, the generic framework activities like planning, communication, modelling, construction, and deployment can be used.
- It will work for the following models :
 - Linear models
 - Iterative and incremental models
 - Evolutionary models and
 - Even for concurrent or component assembly models.
- The process framework is invariant and serves as the basis for all software work performed by a software organization.
- But the actual work tasks do always vary. The process decomposition begins when the project manager asks, "how do we accomplish this framework activity?"
- **For example :** A small, relatively simple project may require the following work tasks for the communication activity :
 - Develop list of clarification issues.
 - Meet with customer to address clarification issues.
 - Jointly develop a statement of scope.
 - Review the statement of scope with all concerned.
 - Modify the statement of scope as required.
- These events may occur in less than 48 hours. These events represent that process decomposition is appropriate for the small and relatively simple project. It should be noted that work tasks must be adapted to the specific needs of the project.

4.6 Observations on Estimation

- The software cost estimation and the efforts estimation can never be the exact. Because there various factors affecting the estimation. **For example** : Manpower, development environment, political environment etc. These factors will affect the overall cost of the project and the efforts taken to develop the project.
- Following are some options available for achieving reliable software project estimation :
 - Try to delay the estimation until the late in the project.
 - Use baseline metrics theory i.e. collect accurate estimates of past similar projects.
 - Use simple decomposition techniques to obtain cost and effort estimates.
 - Use more than one estimation models.
- Software project estimation is a form of problem solving, and in most cases, the problem to be solved is too complex. For this reason, we decompose the problem, re-characterizing it as a set of smaller problems.

4.6.1 Software Sizing

- The accuracy of a software project estimate is predicated on a number of things :
 - The degree estimation for the size of the software to be developed.
 - The ability to translate the estimation for the size into the human efforts, time and money.
 - The abilities of the development team reflected in the project plan and
 - The product stability requirements and the environment that supports the software development efforts.
- Four different approaches to the sizing problem have been suggested :
 - **Fuzzy logic sizing** : In order to apply the fuzzy logic approach, the planner should identify the type of software and should establish its magnitude on a qualitative scale. Later on, this magnitude is refined within the original range.
 - **Function point sizing** : In this, the planner establishes the estimates of the information domain characteristics.
 - **Standard component sizing** : In standard component sizing approach, the software consist of different standard components. Examples of the standard components for an information system are :

1. Subsystems	2. Modules
3. Screens	4. Reports
5. Interactive programs,	6. Batch programs, files LOC
7. Object-level instructions.	
 - The project planner should estimate the number of occurrences of each of the standard components.
 - **Change sizing** : In this approach, the planner should estimate different modifications that is accomplished. This particular approach is generally used when a project uses some existing project and requires its modification.
- Results of each of these sizing approaches should be combined statistically to create a three-point or expected-value estimate.

4.6.2 Problem-based Estimation

- The LOC and FP data are generally used during software project estimation the following two ways are required :
 - As an estimation variable that will enable the sizing of each element of the software and
 - Secondly, as a baseline metrics used to develop cost and effort estimates.
- The project planner starts with a software scope that is a bounded statement, and tries to decompose the software into problem functions. These problem functions can be estimated individually. Later the LOC or FP is estimated for each function.
- Following are baseline productivity metrics :
 - LOC/person-month and
 - FP/person-month
- These baseline productivity metrics are applied to the appropriate estimation variable and from it the cost or effort for the function is derived. Finally the function estimates are integrated to produce one complete estimate for the whole project.
- The expected-value or three-point value can be computed for the estimation variable size S , as a weighted average of the optimistic size (S_{opt}), most likely size (S_m), and pessimistic size (S_{pess}) estimates. Consider the following example,

$$S = \frac{(S_{opt} + 4S_m + S_{pess})}{6} \quad \dots(4.6.1)$$

- The Example (4.6.1) gives the emphasis on “most likely” estimate and it follows a beta probability distribution.

4.6.3 LOC-based Estimation

University Question

Q. Compare Lines of code (LOC) and Function Point (FP) based estimation of software.

SPPU - May 19, 8 Marks

- As an example of LOC and FP problem-based estimation techniques, consider a software package to be developed for a computer-aided design application for mechanical components.
- The software is to execute on an engineering workstation and must interface with various peripherals including a mouse, digitizer, high-resolution colour display, and laser printer. A preliminary statement of software scope can be developed.

Table 4.6.1 : Estimation Tables for the LOC Methods

Function	Estimated LOC
User Interface and Control Facilities (UICF)	2,300
Two-Dimensional Geometric Analysis (2DGA)	5,300
Three-Dimensional Geometric Analysis (3DGA)	6,800
Database Management (DBM)	3,350
Computer Graphics Display Facilities (CGDF)	4,950
Peripheral Control Function (PCF)	2,100



Function	Estimated LOC
Design Analysis Modules (DAM)	8,400
Estimated lines of code	33,200

- For our purposes, we assume that further refinement has occurred and that the major software functions listed in Table 4.6.1 are identified. The decomposition technique for LOC, an estimation table, shown in Table 4.6.1, is developed. A range of LOC estimates is developed for each function.
- For example :** The range of LOC estimates for the 3D geometric analysis function is optimistic 4600 LOC, most likely 6900 LOC, and pessimistic 8600 LOC. Applying Equation (4.6.1), the expected value for the 3D geometric analysis function is 6800 LOC.

4.6.4 FP-based Estimation

University Questions

- Q. Explain the FP based estimation technique.
- Q. Compare Lines of code (LOC) and Function Point (FP) based estimation of software.
- Q. How do you calculate FP and how it is used in estimation of a software project.

SPPU - May 18, 6 Marks

SPPU - May 19, 8 Marks

SPPU - Dec. 19, 5 Marks

Table 4.6.2 : Estimating information domain values

Information domain value	Opt.	Likely	Pess.	Est. count	Weight	FP count
Number of external inputs	20	24	30	24	4	97
Number of external outputs	12	15	22	16	5	78
Number of external inquiries	16	22	28	22	5	88
Number of internal logical files	4	4	5	4	10	42
Number of external interface files	2	2	3	2	7	15
Count total						320

- Decomposition for FP-based estimation focuses on information domain values rather than software functions.
- Referring to the table presented in Table 4.6.2, the project planner estimates external inputs, external output external inquiries, internal logical files, and external interface files for the CAD software.
- FP are computed using the technique discussed. For the purposes of this estimate, the complexity weighting factor is assumed to be average. Table 4.6.2 presents the results of this estimate.

Each of the complexity weighting factors is estimated and the value adjustment factor is computed.

Table 4.6.3

	Factor	Value
1.	Backup and recovery	
2.	Data communications	4
3.	Distributed processing	2
4.	Performance critical	0
		4

Factor		Value
5.	Existing operating environment	3
6.	On-line data entry	4
7.	Input transaction over multiple screens	5
8.	ILFs updated online	3
9.	Information domain values complex	5
10.	Internal processing complex	5
11.	Code designed for reuse	4
12.	Conversion/installation in design	3
13.	Multiple installations	5
14.	Application designed for change	5
Value adjustment factor		1.17

Finally, the estimated number of FP is derived :

$$FP_{\text{estimated}} = \text{count} - \text{total} \times [0.65 + 0.01 \sum (F_i)]$$

$$FP_{\text{estimated}} = 375$$

The organizational average productivity for systems of this type is 6.5 FP/pm.

4.6.5 Process-based Estimation

- The most common technique for estimating a project is to base the estimate on the process that will be used. That is, the process is decomposed into a relatively small set of tasks and the effort required to accomplish each task is estimated.

Table 4.6.4 : Process-based estimation table

Activity →	CC	Planning	Risk analysis	Engineering		Construction release		CE	Totals
Task →				Analysis	Design	Code	Test		
Function ↓									
UICF	-	-	-	0.50	2.50	0.40	5.00	n / a	8.40
2 DGA	-	-	-	0.75	4.00	0.60	2.00	n / a	7.35
3DGA	-	-	-	0.50	4.00	1.00	3.00	n / a	8.50
CGDF	-	-	-	0.50	3.00	1.00	1.50	n / a	6.00
DBM	-	-	-	0.50	3.00	0.75	1.50	n / a	5.75



Activity →	CC	Planning	Risk analysis	Engineering		Construction release		CE	Totals
PCF	-	-	-	0.25	2.00	0.50	1.50	n/a	4.25
DAM	-	-	-	0.50	2.00	0.50	2.00	n/a	5.00
Totals	0.25	0.25	0.25	3.50	20.50	4.50	16.50		46.00
% effort	1%	1%	1%	8%	45%	10%	36%		

CC = Customer Communication,

CE = Customer Evaluation

- A series of framework activities must be performed for each function. Functions and related framework activities may be represented as part of a table similar to the one presented in Table 4.6.4.
- After the problem functions and development process activities are declared, the planner may begin estimating the effort like person-months. This estimation is necessary to accomplish each of the software process activity for each of software function. All these data constructs the central matrix of the table as shown in Table 4.6.4. The average labour rates like cost/unit effort are applied to the effort estimated for each of the process activity.

4.6.6 An Example of Process-based Estimation

- To explain the use of process-based estimation, we again consider the CAD software. Refer the completed process-based table shown in Table 4.6.4, estimates of effort (in person-months) for each software engineering activity are provided for each CAD software function. The engineering and construction release activities are subdivided into the major software engineering tasks.
- Generally the gross estimates of effort are prepared for communication with the customers, planning and risk analysis. It must be noted that nearly 53 percent of the efforts is spent on front-end engineering tasks like requirements analysis and design. It actually indicates the significance and importance of this work.
- **For example :** Let the average burdened labour rate of \$8,000 per month, then the total estimated project cost is computed as \$368,000, and the estimated effort is approximately 46 person-months. In this case, the labour rates could be associated with each framework activity, if desired. It can also be associated with each of the software engineering task and calculated separately.

4.6.7 Estimation with Use-Cases

- Use-cases give software team members to see into insight of the software scope and requirements. But use-case estimation approach can be problematic due to following reasons :
 - Use-cases may be written by using various formats and styles. So far, there is no standard format.
 - They represent the external view of the software and may be described at various levels of abstraction.
 - They do not look at the complexity of the functions and features.
 - And use-cases do not explain complex behaviour also.
 - In contrast to a LOC or a FP computations, the efforts required to implement one person's "use-case" and some other person's "use-case" may vary drastically. That is, one person may require months and another person may require a day or two.

To illustrate how this computation might be made, consider the following relationship :

$$LOC_{estimate} = N \times LOC_{avg} + [(S_a/S_h - 1) + (P_a/P_h - 1)] \times LOC_{adjust} \quad \dots(4.6.2)$$

where, N = Actual number of use-cases

LOC_{avg} = Historical average LOC per use-case for this type of subsystem.

LOC_{adjust} = Represents an adjustment based on of LOC_{avg}

S_a = Actual scenarios per use-case

S_h = Average scenarios per use-case for this type of subsystem

P_a = Actual pages per use-case

P_h = Average pages per use-case for this type of subsystem

- The Equation (4.6.2) is useful in developing the rough estimate of the number based on LOC and the actual number of use-cases used by several scenarios along with the page length of the use-cases. Thus the adjustment shows the historical average LOC per use case.

4.6.8 An Example of Use-Case based Estimation

- The CAD software is composed of three subsystem groups :

1. User interface subsystem
2. Engineering subsystem group
3. Infrastructure subsystem group

- Consider an example in that six use-cases describe the **user interface subsystem**. Each of the use case is written by approximately 10 scenarios and it has an average length of six pages.
- The **engineering subsystem group** is illustrated by approximately 10 use-cases. And each of these use-cases may have 20 scenarios associated with it and it has an average length of nearly eight pages.
- Finally, the **infrastructure subsystem group** is covered by five use-cases with an average of only six scenarios and an average length of five pages.

Table 4.6.5 : Use-case estimation

	Use-cases	Scenarios	Pages	Scenarios	Pages	LOC	LOC estimate
User interface subsystem	6	10	6	12	5	560	3,366
Engineering subsystem group	10	20	8	16	8	3100	31,233
Infrastructure subsystem group	5	6	5	10	6	1650	7,970
Total LOC estimate							42,568

- Using the relationship noted in Equation (4.6.2), the table shown in Table 4.6.4 is developed. Hence the LOC estimate for the user interface subsystem is computed using Equation (4.6.2). Using the same approach, estimates are made for both the engineering and infrastructure subsystem groups. Table 4.6.5 summarizes the estimates and indicates that the overall size of the CAD software is estimated at 42,500 LOC.

4.6.9 Reconciling Estimates

- The estimation techniques discussed in the preceding sections result in multiple estimates which must be reconciled to produce a single estimate of effort, project duration, or cost. To illustrate this reconciliation procedure, we again consider the CAD software.
- Total estimated effort for the CAD software range from a low of 46 person-months (derived using a process-based estimation approach) to a high of 68 person-months (derived with use-case estimation). The average estimate is 56 person-months. The variation from the average estimate is approximately 18 percent on the low side and 21 percent on the high side. The planner must determine the cause of divergence and then reconcile the estimates.

4.7 Object Point (OP)-based Estimation (Estimation of Object-Oriented Projects)

- Estimation for the projects that has been designed especially for object oriented environment is appended with some extra guidelines.
- So object oriented estimation uses the same approach as it was used in conventional software cost estimation, but in addition following approach is suggested :
 - Develop estimates using effort decomposition, FP analysis, and any other method that is applicable for conventional applications.
 - Using object oriented analysis modelling, develop use case and determine a count.
 - From the analysis model, determine the number of classes key
 - Categorize the type of interface for the application and develop a multiplier for support classes as shown as follows :

Interface Type	Multiplier
No GUI	3.0
Text based Interfaces	2.45
GUI	3.5
Complex GUI	4.0

- Now multiply number of key classes obtained in step 3 by the multiplier depicted above to estimate support classes.
 - Multiply total number of classes i.e. key + support of work-units per class usually 15 to 20 person-days per class.
 - Finally cross check the class-based estimate by multiplying the average number of work-units per use case.

4.8 Empirical Estimation Models

- An estimation model for software products uses empirically derived formulas in order to predict the effort in terms of functions of LOC or FP. The values for LOC or FP can be estimated. Here, the resultant values for LOC or FP are used in the estimation model without the values using the tables.
- The empirical data are always supported in most of the estimation models. These empirical data are derived from some sample of projects. Due to this reason only, there is no estimation model suitable for all the classes of software and in all development environments. And hence the results obtained in such cases are used judiciously.

- An estimation model must be calibrated in such a way that reflects its local conditions. The estimation model must be thoroughly tested by applying the data gathered from the past completed projects.

4.8.1 The Structure of Estimation Models

- The estimation model is generally derived from regression analysis on the data that are collected from previously collected software projects. The structure of estimation models take the following form :

$$E = A + B \times (e_v)^c \quad \dots(4.8.1)$$

- Where A, B, and C are nothing but the empirically derived constants. The constant E is an effort in person-months, and the estimation variable is e_v .
- In addition to the relationship described by Equation (4.8.1), most of estimation models use project adjustment component that enables the effort E. Generally the effort E is adjusted by the following project characteristics :
 - Problem complexity,
 - Staff experience,
 - Development environment etc.

- Various LOC-oriented estimation models that have been proposed in the literature are as follows :

- 1) $E = 5.288 \times (KLOC)^{1.047}$ (Doty model for $KLOC > 9$)
- 2) $E = 3.2 \times (KLOC)^{1.05}$ (Boehm simple model)
- 3) $E = 5.2 \times (KLOC)^{0.91}$ (Walston-Felix model)
- 4) $E = 5.5 + 0.73 \times (KLOC)^{1.16}$ (Bailey-Basili model)

Following are some FP-oriented models proposed :

- 1) $E = -37 + 0.96 FP$ (Kemerer model)
- 2) $E = -91.4 + 0.355 FP$ (Albrecht and Gaffney model)
- 3) $E = -12.88 + 0.405 FP$ (small project regression model)

- After observing all these models, we come to conclusion that different results are possible for the same values of LOC or FP.

4.8.2 The COCOMO II Model

University Question

Q. Explain COCOMO model for project estimation with suitable example.

SPPU - Dec. 13, Dec. 15, May 16, Dec. 16, Dec. 18, 8 Marks

- Barry Boehm has devised a software estimation models hierarchy having the name as COCOMO i.e. COncstructive COst MOdel. The COCOMO model has been evolved into more comprehensive model called COCOMO II. It is actually having a hierarchy of estimation models that focus the following areas :
 - **Application composition model** : It is used in the early stages of development, when user interfaces, system interaction, performance assessment and technology evaluation are prime important.
 - **Early design stage model** : Once the requirements are stabilized and basic architecture is constructed, then early design stage model is used.



- **Post-architecture stage model** : It is used during development of the software.
- COCOMOII models also require sizing information like other estimation models for the software. Following three sizing options are available :
 - Object points
 - Function points and
 - Lines of source code
- The COCOMO II model uses object points that are an indirect software measure. They are computed using the counts of :
 - Screenshots taken at user interface
 - Reports and
 - Components required in developing the application.
- The screenshots or the reports are classified into any of the following three complexity levels :
 - Simple
 - Medium
 - Difficult
- The client and server data tables are required to create the screenshots and reports. The complexity is defined as the function of these reports.

Table 4.8.1 : Complexity weighting for object types

Object type	Complexity weight		
	Simple	Medium	Difficult
Screen	1	2	3
Report	2	5	8
3GL component			10

- After the determination of complexity, the number of screenshots, reports, and components object point are weighted as per the Table 4.8.1. In component-based development when software reuse is implemented, then the percent of reuse (% reuse) is estimated and the object point count is adjusted according to the following equation :

$$NOP = (\text{object points}) \times [(100 - \% \text{ reuse})/100]$$

where NOP → New Object Points.

Table 4.8.2 : Productivity rate for object points

Developer's experience/capability	Very low	Low	Nominal	High	Very high
Environment maturity / capability	Very low	Low	Nominal	High	Very high
PROD	4	7	13	25	50

- In order to derive the estimate of effort, a "productivity rate" should be derived first. They are based on the computed NOP value. The Table 4.8.2 shown above presents the productivity rate.
- These productivity rates are illustrated for various levels of developer's experience as well as various levels of environment maturity.

$$\text{PROD} = \text{NOP}/\text{person-month}$$

- After determining the productivity rate, an estimate of project effort can be derived as follows :

$$\text{Estimated effort} = \text{NOP}/\text{PROD}$$

4.8.3 The Software Equation

- The software equation is a model that considers a specific distribution of effort over the project development life. The software equation model is a multivariable model. This model is derived from productivity data collected from nearly 4000 contemporary software projects. Depending on all these data, the estimation model takes the following form :

$$E = [\text{LOC} \times B^{0.333}/P]^3 \times (1/t^4) \quad \dots(4.8.2)$$

Where, E = Person-months effort or person-years efforts

t = Duration of the project in months or years

B = The special skills factor

P = Productivity parameter

- The productivity parameter reflects the following characteristics :
 - Process maturity
 - Management practices
 - Good software engineering practices
 - Programming languages used
 - Software environment
 - The skill set of team members
 - The experience of team members, and
 - The complexity of the software
- Typical values might be P = 2000 for development of real-time embedded software;
- P = 10,000 for telecommunication and systems software; P = 28,000 for business systems applications. The productivity parameter is derived using historical data collected from previous development efforts.
- The software equation has following two independent parameters :
 - An estimate of size and
 - An indication of project duration.

4.9 Requirements Management : Preparing Requirement Traceability Matrix

University Question

SPPU - May 14, 6 Marks

Q. Draw and explain the traceability table for requirement management.

- Requirement management is a software engineering task that helps the project team members to identify the requirements, control the requirements, track the requirements and changes to requirements at any time as the project exceeds.



- The requirement management task starts with identification and each of the requirements is assigned a unique identifier.
- After the requirements finalization, the traceability table is developed. Traceability table relates the requirements to one or more aspects of the system or its environment. The traceability tables are used as requirements database and are useful in understanding how change in one particular requirement will affect other parts or aspects of the system.
- Following are some examples of traceability table :
 - **Features traceability table** observes product features and make sure that how it is closely related to customer requirement.
 - **Source traceability table** is used to identify the source of each of the requirement.
 - **Dependency traceability table** observes the relationships among requirements.
 - **Subsystem traceability table** classifies the requirements as per the subsystem they govern.
 - **Interface traceability table** indicates the internal and external system interface relate as per the given requirement.

Table 4.9.1 : Generic traceability table

Requirement	Specific aspect of the system or its environment						
	A01	A02	A03	A04	A05		All
R01			✓		✓		
R02	✓		✓				
R03	✓			✓			
R04		✓			✓		✓
R05	✓	✓		✓			
							✓
Rnn	✓		✓				

Review Questions

1. Explain the quality attributes, considered in software design
2. What are the categories of software engineering resources ?
3. Explain LOC-based estimation.
4. Explain COCOMO model for project cost estimation.
5. Explain decomposition techniques.
6. Draw and explain the traceability table for requirement management.

5

Unit - III

PROJECT SCHEDULING

Syllabus

Project Scheduling : Project Scheduling, Defining a Task for the Software Project, Scheduling.
Suggested Free Open Source Tools : Grantt Project, Agantty, Project Libre.

5.1 Project Scheduling

University Question

Q. What is project scheduling? What are the basic principles of project scheduling ?

SPPU - May 12, Dec. 16, Dec. 19, 8 Marks

- Project scheduling involves separating the total work involved in a project into separate activities and judging the time required by these activities.
- Scheduling in software engineering can be considered from two views. First, end date for release has been established for a project and organization is responsible for distributing effort within prescribed time frame. Secondly, rough chronological wounds are discussed and end date is set by organisation.
- Thus, project scheduling is an activity that distributes estimated effort across the planned project duration by specific allocation of effort to the specific task in software engineering.
- Basic principles for software project scheduling are :
 1. **Compartmentalization** : Project is divided or compartmented into number of manageable activities, actions and tasks.
 2. **Interdependency** : Interdependency between each compartmentalized activity, action task must be determined.
 3. **Time allocation** : Each task is assigned some work unit i.e. effort by person in days.
 4. **Effort Validation** : Every project has defined number of people and as time allocation occurs, project manager should check no more than allocated numbers of people have been scheduled at any given time.
 5. **Defined Responsibilities** : Every task in schedule should be assigned to a specific team member.
 6. **Defined Outcomes** : Every schedule task should have defined outcome i.e. work product or a part of work product.
 7. **Defined Milestones** : Every task or group of tasks is associated with project milestone.



5.1.1 Defining a Task Set for the Software Project

University Question

SPPU - Dec. 18, May 19, 8 Marks

Q. What is a task network in project scheduling ? Explain with an example.

- Regardless of model used, process model is populated by a set of tasks that enable a software team to define, develop and ultimately support computer software. But the same set of tasks is not appropriate for all types of projects.
- For larger projects, different set of tasks will be applicable. In the same way, for complex projects also, some different set of tasks will be used.
- For developing a project schedule, the entire task set should be divided on the project time line.
- For each project, the set of tasks will vary depending on the type of the project.
- Following are some types of projects :
 - Concept development projects
 - New project having certain application
 - Application enhancement projects
 - Maintenance projects
 - Reengineering projects.
- In a particular type of project, many factors influence the task set to be chosen.
 - These factors include :
 - Size of the project
 - Number of users
 - Stability requirements
 - User friendliness
 - Ease of communication between the application developer or user
 - Performance
 - Technology used.

Example :

- Consider the example of concept development projects. Following are the sets of tasks those can be applied :
 - Concept scope
 - Concept planning
 - Risk assessment and management
 - Proof of concept
 - Implementation
 - Customer feedback and reaction.
- These are certain activities or the set of tasks those can be applied for the concept development projects.

5.1.2 Scheduling

University Question

Q. What is time line chart ? Explain with suitable example.

SPPU - Dec. 12, May 18, 8 Marks

- Scheduling of different engineering activities can use the available project scheduling tools and techniques.
- Following are some project scheduling tools and techniques :
 - PERT (Program Evaluation and Review Technique)
 - CPM (Critical Path Method)
- These are two project scheduling methods that can be applied to the software development process.
- Both tools use the data and information received from the earlier developments.
- Both tools allow to determine the critical path, duration of the projects, and time estimate for the individual activity, efforts taken, duration.

Time-line charts

- A time-line chart, also called Gantt chart, is generated on the basis of the start date inputs for each task.
- Fig. 5.1.1 shows an example Gantt chart. It elaborates the project schedule.
- In the left hand column of the Fig. 5.1.1, all the project tasks are listed. The horizontal lines indicate the duration of the task.
- For concurrent activities, multiple horizontal lines are used.

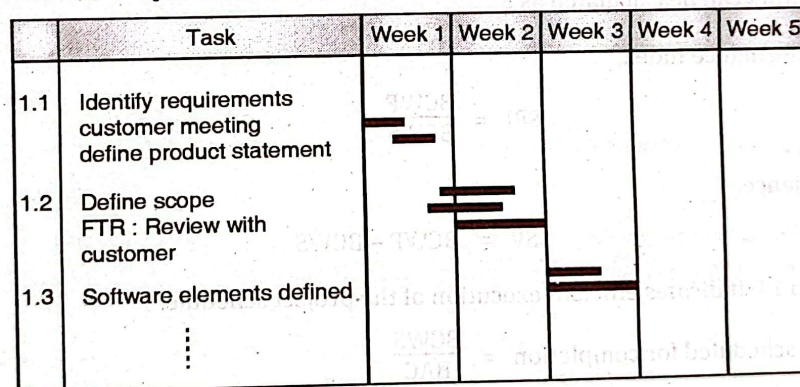


Fig. 5.1.1 : An example time-line chart

5.1.3 Tracking the Schedule

- The project schedule is nothing but a road map that defines the tasks and the milestones to be tracked.
- Tracking is achieved in different ways as follows :
 - Conduct periodic meeting to find out the progress status and problems.
 - Evaluate the process.
 - Check whether all the activities are completed within the deadline or by the schedule date.
 - Compare the planned date and actual start date for each activity.
 - Meet practitioners informally to assess the progress.
- All these tracking techniques are used by the project managers.



5.2 Earned Value Analysis

University Question

Q. Explain the earned value analysis in project scheduling.

SPPU - May 16, May 18, 8 Marks

- During the project tracking, each developer gives his project or task progress status to manager. But this assessment of information is subjective.
- There should be some quantitative technique for assessing progress of project.
- Earned Value Analysis (EVA) is a technique used for quantitative analysis in the following manner :
The EVA provides a common value scale for every task. The time to complete total project is estimated and every task is given an earned value based on its estimated percentage of the total.
- To determine the earned value, following steps are performed :
 1. The Budgeted Cost of Work Scheduled (BCWS) is calculated for each task.
 2. The BCWS values for all work tasks are summed to calculate the final budget.

$$\therefore BAC = \sum(BCWS_k) \text{ for all tasks } k.$$

Where BAC = Budget at Completion.

3. Now, the value for Budgeted Cost of Work Performed (BCWP) is computed. The value of BCWP is sum of the BCWS values for all work tasks completed by that point in time.
- The progress indicators can be calculated as :
 - Scheduled Performance Index,

$$SPI = \frac{BCWP}{BCWS}$$
 - Scheduled Variance,

$$SV = BCWP - BCWS$$
 - An SPI value close to 1.0 indicates efficient execution of the project schedule.

$$\text{Percent scheduled for completion} = \frac{BCWS}{BAC}$$
 - It provides the percentage of work that should have been completed by time t.

$$\text{Percent complete} = \frac{BCWP}{BAC}$$
 - It provides the percent of completeness of the project at a given point in time t.
 - Also, the Actual Cost of Work Performed (ACWP) is the sum of the efforts actually expended on work tasks that have been completed by a point in time on the project schedule.

$\therefore$ Cost performance index,

$$CPI = \frac{BCWP}{ACWP}$$

$$\text{Cost variance, } CV = BCWP - ACWP$$

- A CPI value close to 1.0 provides a strong indication that the project is within its defined budget.

5.3 Schedule Tracking Tools

- The project planning is an important activity performed by the project managers in order to estimate the following entities for their optimum use :
 - The resources required for the project under development.
 - The cost of the project and
 - The schedule for in-time completion of the project.
- Project Managers can use a range of tools and techniques to develop, monitor and control project schedules.
- These tracking tools can automatically produce critical path diagrams or Gantt charts. These tools also instantly update time plans as soon as new information is entered and produce automatic reports to monitor and control the project.
- These tracking tools can assist in planning and controlling the expenditures more effectively and improve communication of financial reporting.
- These tools also provide automatic report template with good GUI to effectively communicate with all the stakeholders of the project.
- These tools also look into work breakdown structure and risk management with greater accuracy and ease of updating and printing the reports.
- Following are some examples of Schedule Tracking Tools :
 - Microsoft Project
 - Daily Activity Reporting and Tracking (DART).

5.3.1 Microsoft Project

- We can use Microsoft Project 2010 or later versions to plan team projects, schedule tasks, assign resources, and track changes to data that is stored in Team Services or TFS.
- By using Project, you can access many tools and functions through the simplified graphical menus and Office Ribbon. The Team tab menu, as shown in the Fig. 5.3.1, displays the same functions that are available from the Team tab in Excel.

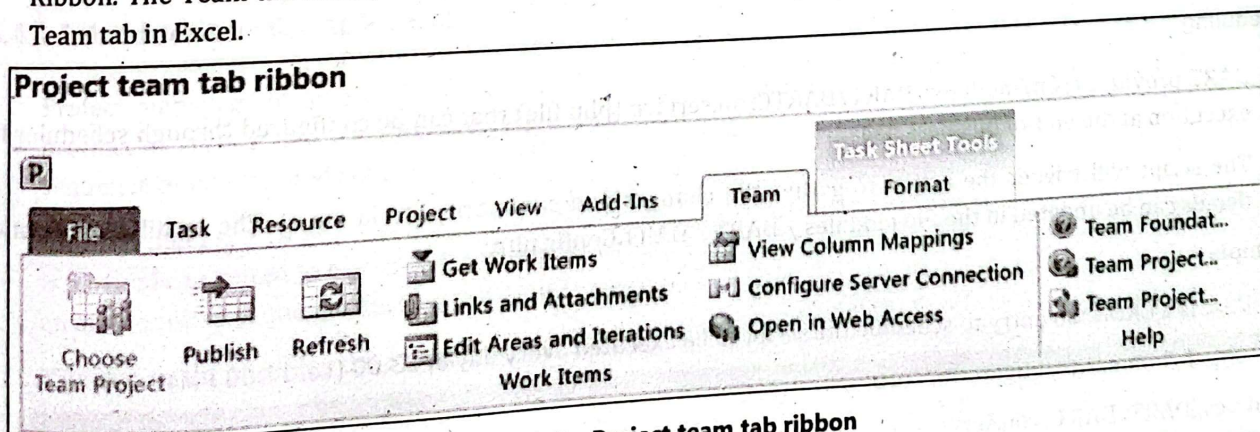


Fig. 5.3.1 : Project team tab ribbon

- Project 2010 and later versions support several new project fields and functions. Depending on how you use Project to schedule team tasks, you may want to update the task work item form to display some of the new fields.

- To maintain new Project fields in both your project plan and in TFS, you have to customize the task work item type and the Microsoft Project Field mapping file for the team project.
- In Project 2010 and Project 2013, New task-related features are facilitated to schedule the tasks manually or automatically.
- By using Task Mode, which is accessed through the following Ribbon menu, you have more flexibility in the way you and team members schedule tasks.
- Start and finish dates for auto scheduled tasks (📅) are determined by the scheduling engine based on task dependencies and the project calendar, as in previous releases of Project.

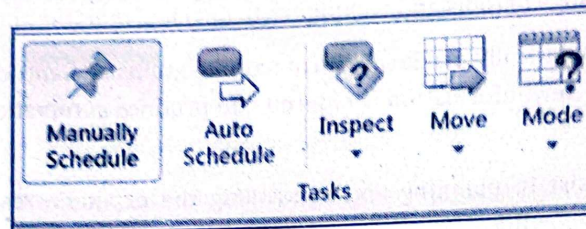


Fig. 5.3.2 : Task Mode

- Project managers who are accustomed to automatic scheduling with past versions of Project can turn the new manual scheduling feature off for specific tasks or the entire project.
- Finally, it is quite simple to use Microsoft Projects for scheduling the projects. It is quite similar to other products of Microsoft office like Microsoft word, Microsoft excel and others.

5.3.2 Daily Activity Reporting and Tracking (DART)

University Question

Q. Write about Daily Activity Reporting and Tracking (DART).

SPPU - Dec. 19, 6 Marks

- Daily Activity Reporting Tool (DART), enables you to track the changes to record being made by users. The extension provides the ability to dispatch the email with summary of changes.
- After installation, in the tool we see various tabs. Here we provide brief introduction of these tabs and their usage.

Scheduling

- DART provides cron/modules/DART/DARTCron.service (php file) that can be configured through scheduler for execution at the end-of business day.
- The script will trigger the action to gather the changes and summarize it via email. The email configuration details can be updated in the file modules / DART / DART.Config.php.

Example

Here is a CRON tab entry to schedule the script to be executed every day at 23:00 (i.e. 11:00 PM)

```
0 23 php -f /home/site/vtigercrm/cron
/modules/DART/DARTCron.service
```

Module Views

When you open DART it shows the list of changes tracked for the day. You can click on Refresh now link if you don't see it updated. The changes are updated when the CRON script is scheduled too.

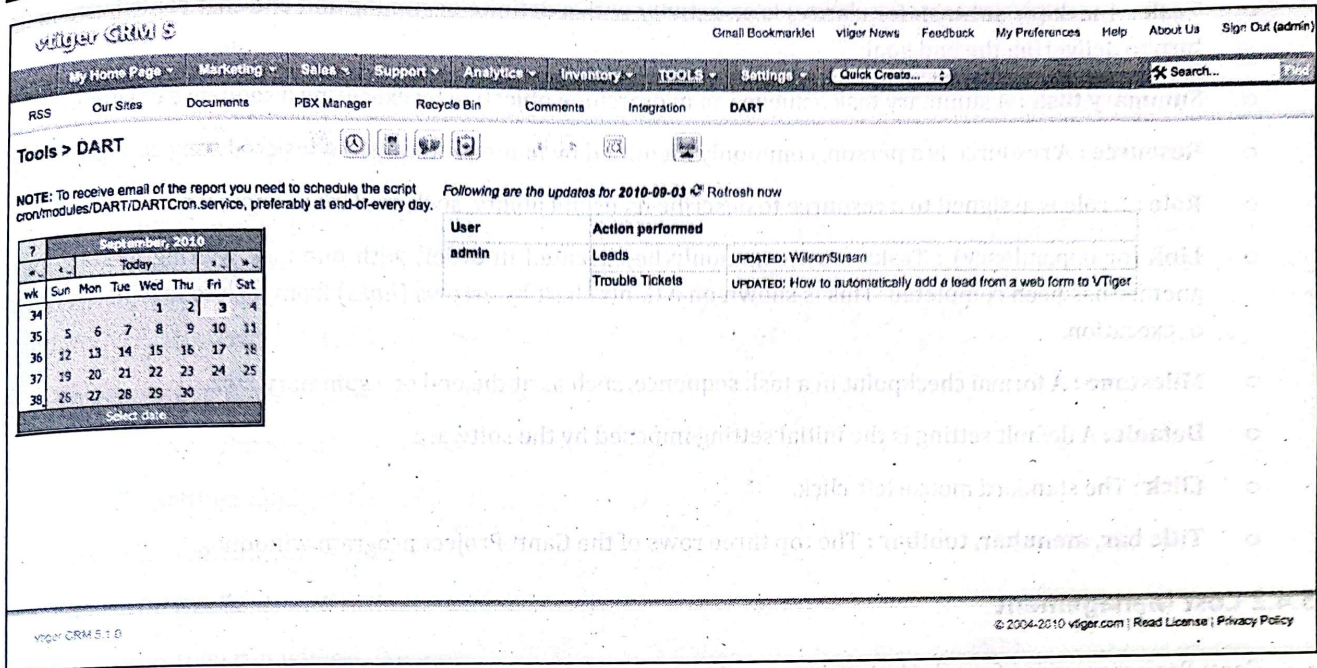


Fig. 5.3.3 : DART Snapshot

No activity notification

- DARTCron.service can send notification emails to group of users who have not used the system for the given day.
- To enable this, you need to uncomment the line in cron/modules/DART/DARTCron.service

```
// $dartCron->sendNoActivityUpdateEmail();
```

As

```
$dartCron->sendNoActivityUpdateEmail();
```

5.4 Case Study - Open Source Tool : Gantt Project

5.4.1 Introduction : Gantt Project

- Project management involves the overview of a team whose members work cooperatively to produce an end product or service that benefits an organisation. The project has an **end goal** that is achieved by completing a sequence of interrelated tasks.
- **Gantt Project** is a software package that allows effective, on-going management of project data. The application is particularly suited to the so-called waterfall model of project management, in which tasks must be executed in an inter-related sequence, leading to a final end goal state.
- This document provides the unfamiliar user with an emerging introduction to *Gantt Project*. It will be somewhat easier if the user has a beginning understanding of project management concepts.

Terminology

- A few terms will appear repeatedly as we progress. It will help if the reader has an understanding of each of these terms. They are :
 - **Project** : A project is a sequence of inter-related tasks that lead to a definite end goal.



- **Task :** A task (or subtask for clarity) is an activity with a definite beginning and end that contributes in its turn to delivering the end goal.
- **Summary task :** A summary task achieves an overarching objective by executing a sequence of subtasks.
- **Resource :** A resource is a person, commonly identified by name, who causes a task to be executed.
- **Role :** A role is assigned to a resource to describe its useful ability, such as *clerk* or *engineer*.
- **Link (or dependency) :** Tasks must commonly be executed in order, with one task waiting to start until another has been completed. This is shown on a Gantt chart by arrows (*links*) from task to task in the order of execution.
- **Milestone :** A formal checkpoint in a task sequence, such as at the end of a summary task.
- **Default :** A default setting is the initial setting imposed by the software.
- **Click :** The standard mouse left-click.
- **Title bar, menubar, toolbar :** The top three rows of the Gantt Project program window.

5.4.2 Cost Management

- Gantt Project provides for calculating the cost of resources (personnel) assigned to each task, and hence the cost of the whole project. The cost of the project, or of any summary task, is the sum of the costs of individual tasks.
- In calculating the cost of an individual task, the contributing factors are :
 - The duration (in days) of the task, which gives the time for which resources are deployed.
 - The daily cost of each resource, being the standard rate from the Resource Payment Rate in Resource Properties (this has no stated currency).
 - The utilization (as a percentage) of a resource's time devoted to the task; this is the unit value found in Gantt tab > task > Task Properties... > Resources tab > unit.
 - The number of resources assigned to the task.

Properties for FD corrections

General Predecessors Resources Custom Columns

Add Delete

ID	Resource Name	Unit	Coordinator	Role
1	A.Minion	75.0	☐	senior clerk
2	B.Minion	35.0	☐	clerk
			☐	

- Task cost -

☒ Calculated: 112.50

☐ Set explicitly:

Ok Cancel

- Costs are calculated automatically by default; the cost is calculated for each resource by :

$$\text{Cost} = \text{duration} \times \text{standard rate} \times \text{unit} / 100$$
- The whole cost of a task is the sum of all individual resource costs (even when there is just one).
- An explicit cost of a task may be typed in at Gantt > Task properties > Resources tab (see above)



- The cost of a summary task is the sum of individual task costs, but can also be set manually.
- There are usually multiple summary tasks in a project. To get a cost for a whole project, just create a single top-level whole project task and indent all tasks one more step under that new one.

5.4.3 Manage Calendar Details

By default, Gantt Project assumes that work on a project takes place, Monday to Friday without weekend work, all year. In reality, work may run for the whole seven-day week (rush job), or Wednesday to Sunday (a restaurant) for example. Further, most communities enjoy public holidays and individual people take leave for a variety of reasons. Gantt Project can allow for all of this. However, Gantt Project only provides for durations measured in whole days.

- **To prepare to manage calendar matters**

Go to *Project > Properties > Calendar*

- The settings apply to the whole project

- **To enable working across weekends**

Choose *On weekends > all tasks run as on other days*.

- Tasks then run without weekend disruption, and minor changes are made to the layout of days on the Gantt chart pane
- See also Gantt Project : Enable weekends as work time

- **To relocate the weekend break to other days of the week**

In the row *Choose weekend*, check the two days which form the break. Uncheck the rest.

- These settings can be varied as you work through the project.

- **To allow for public holidays**

From the row *Holiday calendar*, choose your relevant list from the pull-down menu.

- You can inspect the selected list just below on the *One-off* and *Recurring* tabs.

- **To allow for an individual resource's leave of absence**

Leave for an individual resource does not apply to the whole project and is handled differently.

Allocation of start dates

When a new task is first added (without details), the new taskbar will appear, starting at the earliest date currently displayed in the Gantt chart pane. The *Begin date* and *End date* will be automatically derived and displayed when links are inserted.

Move project start date

In *Project > Properties > Calendar* there is an option called *Move project start date* (near the bottom of the dialog box). Use this setting to change the whole project to a different starting date.

Import holiday calendars in iCalendar format

- The placement of public holidays in the calendar is quite arbitrary and changes from year to year. Assembling a world-wide list is an arduous task. Many individual calendars are now available online in iCalendar format published on Mozilla Holiday Calendars or iCalShare web sites. These calendars can be imported into GanttProject.

- The procedure is :
 1. Find the required calendar online
 2. Download the iCalendar as a .ICS file
 3. In GanttProject, use *Project > Import* to import the .ICS file.

Use a neutral day

Occasionally it may be necessary to designate a day as *neutral*. This day will then not be considered in any GanttProject analysis. To set a day as neutral, choose *Project > Properties > Calendar > One-off > Add* (enter date) > *Type* (choose *neutral*). You can also choose a reminder colour that appears in the timeline charts.

Review Questions

1. What is project scheduling?
2. What are the basic principles of project scheduling?
3. Define time line chart ?
4. How time line chart is used in scheduling of software project ?
5. Write short note on : Schedule Tracking Tools.
6. Write short note on : Daily Activity Reporting and Tracking (DART).
7. Explain the earned value analysis in project scheduling.